

SymPy Bug Report

1 Bug 46: Matrix.is_diagonalizable Ignores Eigenvalue-Collision Branch

1.1 Minimal Reproducer

```
from sympy import Matrix, symbols

a, b = symbols("a b")
M = Matrix([[a, 1], [0, b]])

print("is_diagonalizable:", M.is_diagonalizable())
P, D = M.diagonalize()
print("P:", P)
print("D:", D)

M_equal = M.subs(b, a)
print("specialized matrix:", M_equal)
print("specialized is_diagonalizable:", M_equal.is_diagonalizable())
```

1.2 Observed Behavior

```
SymPy version: 1.14.0
is_diagonalizable: True
P: Matrix([[1, -1/(a - b)], [0, 1]])
D: Matrix([[a, 0], [0, b]])
specialized matrix: Matrix([[a, 1], [0, a]])
specialized is_diagonalizable: False
```

SymPy reports that the parametric matrix is diagonalizable without qualification. The accompanying diagonalization matrix contains the denominator $a - b$, and the specialization $b = a$ is separately recognized by SymPy as non-diagonalizable. Thus the unconditional answer `True` ignores an exceptional parameter branch on which the stated property fails.

1.3 Expected Behavior

The result should be conditional, unknown, or otherwise qualified: for

$$M = \begin{pmatrix} a & 1 \\ 0 & b \end{pmatrix},$$

the matrix is diagonalizable when $a \neq b$, but it is not diagonalizable when $a = b$. An unconditional `True` from `M.is_diagonalizable()` is therefore mathematically incorrect.

1.4 Mathematical Explanation

For $a \neq b$, the upper-triangular matrix has two distinct eigenvalues, a and b , so it is diagonalizable over a field containing the parameters. The eigenvector reported for the eigenvalue b is proportional to

$$\begin{pmatrix} -1/(a-b) \\ 1 \end{pmatrix},$$

which is valid only under the side condition $a - b \neq 0$.

On the collision branch $b = a$, the same matrix becomes

$$M_{b=a} = \begin{pmatrix} a & 1 \\ 0 & a \end{pmatrix} = aI + \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}.$$

For the only eigenvalue a ,

$$M_{b=a} - aI = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix},$$

whose nullspace is one-dimensional. The algebraic multiplicity of a is 2, but the geometric multiplicity is 1, so $M_{b=a}$ is a nontrivial Jordan block and is not diagonalizable. The generic diagonalization is not an equivalent representation on this branch, because the change-of-basis matrix contains $1/(a-b)$ and is undefined when $a = b$.

1.5 Source-Level Diagnosis

The relevant source file is `sympy/matrices/eigen.py`. The diagnosis narrows the issue to its eigenvector and diagonalizability pipeline. The public diagonalizability query dispatches through the `MatrixBase` method to `_is_diagonalizable`. That routine calls the helper `_is_diagonalizable_with_eigen`. The same helper is also used by `diagonalize()` through `_diagonalize`, after which the returned eigenvectors are used to build P and D .

The relevant helper computes `M.eigenvecs(simplify=True)` and then checks each listed eigenvalue entry independently. For the reproducer, `eigenvecs()` reports one eigenvector for eigenvalue a and one eigenvector for eigenvalue b , with the latter containing the denominator $a - b$. The check compares each listed eigenvalue's algebraic multiplicity with the number of eigenvectors reported for that listed key. It does not track the condition $a \neq b$, and it does not revisit the branch where the two symbolic eigenvalue keys collide.

```
(a, 1, [Matrix([[1], [0]])])
(b, 1, [Matrix([[-1/(a - b)], [1]])])
```

This source-level behavior explains the mathematical failure: the generic eigenspace calculation treats $a - b$ as invertible, so `_is_diagonalizable_with_eigen` accepts a decomposition that is valid only away from the exceptional branch. When $b = a$, the two eigenvalue entries collapse into a single eigenvalue of algebraic multiplicity 2, but the eigenspace has dimension 1. The diagnosis identifies a plausible fix direction: symbolic diagonalization should carry parameter conditions from eigenvalue distinctness and from denominators introduced while computing eigenvectors; at minimum, when symbolic eigenvalue equality is undecidable, `is_diagonalizable` should avoid returning unconditional `True` without assumptions such as $a \neq b$.

1.6 Additional Failing Instances

The independent checks exercise the same family with nonzero upper-right entries. For

$$M_c = \begin{pmatrix} a & c \\ 0 & b \end{pmatrix} \quad (c \neq 0),$$

the generic matrix is diagonalizable when $a \neq b$, but the branch $b = a$ gives

$$\begin{pmatrix} a & c \\ 0 & a \end{pmatrix},$$

which has a one-dimensional eigenspace for the repeated eigenvalue a . The expected behavior is the same for every nonzero c : SymPy should not return an unconditional **True** for the parametric matrix while the $b = a$ specialization is defective.

Input / Expression	SymPy behavior	Expected behavior
$M = \begin{pmatrix} a & 1 \\ 0 & b \end{pmatrix}$	Generic True ; specialized False	Not unconditional True ; defective at $b = a$
$M = \begin{pmatrix} a & 2 \\ 0 & b \end{pmatrix}$	Generic True ; specialized False	Not unconditional True ; defective at $b = a$
$M = \begin{pmatrix} a & 3 \\ 0 & b \end{pmatrix}$	Generic True ; specialized False	Not unconditional True ; defective at $b = a$
$M = \begin{pmatrix} a & 4 \\ 0 & b \end{pmatrix}$	Generic True ; specialized False	Not unconditional True ; defective at $b = a$
$M = \begin{pmatrix} a & 5 \\ 0 & b \end{pmatrix}$	Generic True ; specialized False	Not unconditional True ; defective at $b = a$
$M = \begin{pmatrix} a & 6 \\ 0 & b \end{pmatrix}$	Generic True ; specialized False	Not unconditional True ; defective at $b = a$

1.7 Related Issues Assessment

The best-effort deduplication check found related SymPy documentation and general linear-algebra references describing diagonalizability, the contract for `diagonalize()`, and non-diagonalizable Jordan-block examples, but it did not identify a public SymPy issue, pull request, Stack Overflow post, mailing-list thread, or release note for this exact $\begin{pmatrix} a & 1 \\ 0 & b \end{pmatrix}$ parameter-collision case.

The verdict is a new instance of a known failure mode: symbolic computations can be valid on a generic parameter branch while failing on exceptional branches, but this concrete SymPy `is_diagonalizable/diagonalize` reproducer appears previously unreported and remains individually actionable as an issue and regression test.

1.8 Proposed SymPy Regression Test

```
import pytest
from sympy import Matrix, symbols

a, b = symbols("a b")

@pytest.mark.parametrize("offdiag", [1, 2, 3, 4, 5, 6])
def test_is_diagonalizable_does_not_ignore_eigenvalue_collision(offdiag):
    M = Matrix([[a, offdiag], [0, b]])
    assert not (M.is_diagonalizable() is True and M.subs(b, a).is_diagonalizable() is
False)
```